

BAZARIX STOREFRONT

Full-Stack (MERN) Architecture and Technical Manual

A detailed technical review of database schemas, API specs, auth, and state workflows.

DOCUMENT METADATA

Release Version:	v1.0.0 (Production-Ready)
Architecture Style:	SPA Frontend (Vite) + REST API Backend (Express)
Storage Layer:	MongoDB Atlas (Mongoose Object Document Mapping)
Generated On:	June 23, 2026
Author / Agent:	Antigravity AI Pair Programming Companion
Security Scope:	Confidential - System Architecture Specifications

Technical Context:

This documentation serves as an overview technical manual for the Bazarix multi-vendor e-commerce platform. It provides insights into the directory layout, Mongoose collection structures, access control logic, payment integrations, and React-Redux frontend design. Use this manual to navigate the repository, debug workflow issues, or design new database entities.

Table of Contents

1. Executive & Architecture Overview	Page 2
2. Project Directory Structure	Page 3
3. Database Schema Design	Page 4
4. Core Features & Role-Based Workflows	Page 6
5. Security, Authentication, & Access Control	Page 7
6. Order Processing & Payment Gateway	Page 8
7. Frontend Architecture & Redux Store	Page 9
8. Appendix: Full API Endpoint Reference	Page 10

1. Executive & Architecture Overview

Bazarix is a state-of-the-art, full-featured multi-vendor e-commerce storefront designed to connect regional artisans, sellers, and general merchants directly with buyers. The platform leverages a modern Single Page Application (SPA) frontend decoupled from a stateless REST API backend, providing rapid page transitions and efficient data transfers. The repository is structured as a Monorepo, dividing codebases into a decoupled frontend client and express server.

Technology Stack Details

- **Frontend:** React 19 + Vite 8. Fast hot-module-reloading, build assets outputting static CSS/JS packages. Utilizes Redux Toolkit for state management, Tailwind CSS for visual aesthetics, and Recharts for analytical widgets.
- **Backend:** Node.js 18+ + Express 4/5. Utilizes middleware layers for platform security (Helmet), server compression, request logging (Morgan), CORS configurations, error handling, and file upload parsing (Multer).
- **Database:** MongoDB (Atlas or Local mongod) with Mongoose ODM. Schemas enforce data consistency, cast object identifiers, implement query middleware, and optimize indices on search-heavy fields.
- **Integrations:** Payment collection uses secure transaction-signing hashes (JazzCash and Easypaisa) with a fallback simulation loop. Media asset persistence utilizes Cloudinary cloud buckets.

2. Project Directory Structure

The workspace directory contains separate root folders for the customer-seller frontend client (my-project) and the REST API server (server). The structure is outlined as follows:

```

multivendor ecommerce store/
|-- my-project/                # React + Vite Client Source
|   |-- src/
|   |   |-- api/              # Axios clients & API request triggers
|   |   |-- components/      # Reusable shell elements (Navbar, ProtectedRoute)
|   |   |-- pages/           # Route layouts structured by user context
|   |   |   |-- admin/       # User/Seller logs, Sales reports, Settings
|   |   |   |-- auth/        # Login, Register, Password recovery UI
|   |   |   |-- customer/    # Order tracking, Checkout, Profile, Wishlist
|   |   |   |-- public/      # Homepage, Shop browser, Product Detail, Cart
|   |   |   |-- seller/      # Seller dashboard, Inventory control, Discounts
|   |   |-- store/           # Redux Toolkit Slices (auth, cart, products)
|   |   |-- index.css         # Core styling variables & Tailwind imports
|   |   +-- main.jsx          # DOM mounting & store provider injection
|   +-- package.json          # Frontend package dependencies
+-- server/                   # Express Server Backend
    |-- api/                  # Route routing definitions
    |-- config/               # Database connection setups
    |-- controllers/          # Core route handler algorithms
    |-- middleware/           # Auth tokens, errors, file validation, roles
    |-- models/               # Mongoose Schema Definitions
    |-- routes/               # Main Express route files
    |-- seed/                 # Database seed scripts (demo products & assets)
    |-- utils/                # Payment helpers, JWT generators, SendMail
    +-- server.js             # Application server endpoint

```

This modular separation allows backend endpoints to change or migrate independently of visual changes on the React client. Production builds output the frontend to static files (under /dist) which are served by platforms like Firebase or Vercel, while the backend server can be run as a serverless instance or a persistent container.

3. Database Schema Design

The storage architecture uses Mongoose schemas to represent collections, enforce field types, model relationship sub-references, and automate document transformations (like turning MongoDB '_id' and version key '___v' into a clean json 'id' attribute).

Core Collection Schema Definitions

User Schema

Stores accounts. Tracks provider ('local' | 'google' | 'facebook'), providerId, role ('admin' | 'seller' | 'customer'), name, email, password (hashed, selected false), phone, city, isApproved (for seller onboarding), isActive (for blacklist/deactivation), address sub-document, resetPasswordToken/resetPasswordExpire.

Indexing details: Indexed on email (unique).

Shop Schema

Stores details of shops owned by sellers. Fields include owner (ObjectId ref User), name (unique), description, logo URL, banner URL, isActive, totalRevenue, totalOrders, and rating. Automatically tracked via timestamps.

Indexing details: Indexed on owner (unique), name (unique).

Product Schema

Stores item inventory details. Fields: seller (ObjectId ref User), shop (ObjectId ref Shop), name, description, price, discountPrice, stock, image (primary thumbnail url), images (gallery array), category, region, culture, rating, numReviews, isPublished, variants (color, size, SKU, stock array), and sellerName.

Indexing details: Text indices on name & description. Compound index on {shop, isPublished}.

Order Schema

Models transactional logs. Fields: customer (ObjectId ref User), orderItems (sub-schema containing product ref, name, qty, price, image, seller/shop refs), shippingAddress, paymentMethod ('Easypaisa' | 'COD' | 'JazzCash' | 'Stripe'), paymentResult (id, status, updateTime, emailAddress), itemsPrice, shippingPrice, taxPrice, totalPrice, orderStatus ('Pending' | 'Processing' | 'Shipped' | 'Delivered' | 'Cancelled'), isPaid, paidAt, isDelivered, deliveredAt, gatewayTxnRef.

Indexing details: Indexed on customer, and compound on orderItems.seller.

Auxiliary Collection Schema Definitions

Category Schema

A simple collection matching product tags. Fields: name (unique), slug (url-safe name generated via a pre-save hook), and icon. Pre-save hook uses a regex to transform spaces and characters into a lowercase kebab-case slug.

Review Schema

Handles rating feedback. Fields: product (ObjectId ref Product), customer (ObjectId ref User), rating (1 to 5), title, and comment. Built with a compound index on {product, customer} with {unique: true} constraints, preventing a customer from submitting multiple reviews on the same product.

Discount Schema

Models seller coupon campaigns. Fields: seller, shop, name, scope ('product' | 'category' | 'shop'), product reference, category tag, type ('percentage' | 'fixed'), value (PKR or percent), active, startsAt, and endsAt. Indexed on {seller, active}.

Wishlist Schema

A customer-specific collection. Fields: customer (ObjectId ref User, unique) and products (array of ObjectIds ref Product). It maps user favorites to allow persistence across browser sessions.

Notification Schema

User notification records. Fields: user (ObjectId ref User), type ('order' | 'payment' | 'system'), title, message, and read (boolean). Sorted and cleaned regularly to show transactional updates.

4. Core Features & Role-Based Workflows

The platform is organized around three distinct user roles, each accessing customized views, forms, and controller boundaries. The workflow of each role is specified below:

1. Customer Portal Experience

- **Browse & Discovery:** Allows public users to browse catalog items, search via queries, filter by price thresholds, categories, cultural regions (e.g. Sindh, Punjab), and view vendor shops.
- **Shopping Cart:** Saves quantities, validates stock in real-time, stores wishlist items, and applies coupons generated by sellers.
- **Checkout Process:** Completes shipping details, selects payment methods (COD or online), redirects to Easypaisa/JazzCash gateways, and records non-sensitive card metadata.
- **Order Logs:** Provides order history logs, tracks item status transitions, displays notifications, and allows writing single product reviews.

2. Seller Operations Dashboard

- **Onboarding:** Registration requires admin approval. Once approved, the system instantiates their custom Shop profile automatically.
- **Inventory:** Full CRUD control over their products, categories, SKU variations, price points, and discount flags.
- **Fulfillment:** Receives order notices containing their items, tracks sub-orders, and triggers status updates ('Processing' to 'Shipped').
- **Analytics & Payouts:** Tracks sales charts via Recharts, manages active discount codes, and requests earnings withdrawals.

3. Platform Administrator Panel

- **Overview Stats:** Aggregated platform revenue metrics, total orders, active user counts, and system-wide products.
- **Merchant Approval:** Reviews pending seller applications, reads shop documentation, and clicks approve/reject (triggering emails & notifications).
- **User Moderation:** Access to user list logs and handles account block/unblock (soft-deactivates accounts in the User collection).
- **Coupons & Settings:** Defines platform-wide categories, configures commission rates, and resolves support tickets.

5. Security, Authentication, & Access Control

Platform security is implemented across multiple layers, including route-level JWT validation, strict request body sanitization, automated role filters, and external social OAuth credential verifications.

JSON Web Token (JWT) Authentication

Authentication uses a stateless JWT model. When a user logs in locally via email/password, the server verifies the password using bcrypt. If correct, a JWT is signed with the user ID, role, and expiration, and returned to the client. The client stores the token in localStorage and includes it in the HTTP 'Authorization: Bearer <token>' header for subsequent requests.

Access Control Middleware Flow

The Express server uses custom middleware layers to restrict endpoints:

- **protect.js:** Extracts the Bearer token, decodes the user payload, fetches the User document from the database (excluding the password field), and attaches it as req.user. Triggers a 401 response on invalid or missing tokens.
- **authorize.js:** Curried middleware that evaluates req.user.role against a whitelist. If req.user.role does not match, a 403 Forbidden error is returned, stopping downstream controller execution.
- **sellerApproval.js:** Ensures that if the user role is 'seller', their isApproved flag is true. This prevents unapproved merchants from accessing seller-level APIs even with a valid token.

OAuth Integration & Account Linking

The authController supports social authentication. For Google Login, the client sends a Google ID Token (credential) verified by the backend via OAuth2Client. For Facebook, the server requests profile details from the Graph API using the client's access token. If the email doesn't exist, a new Customer account is created. If the email exists, the account links the social provider credentials automatically.

Secure Password Recovery Loop

When a password reset is requested, a secure, short-lived (15 min) token is generated. The SHA-256 hash of this token is saved on the user document (resetPasswordToken), while the raw hex string is sent via Nodemailer to the user's email. When the user submits a new password, the raw string is hashed and matched against the database records.

6. Order Processing & Payment Gateway

The order and payment flow represents the main transactional core of the Bazarix platform. The database records, checkout math, and gateway redirects must remain consistent to prevent fraud.

Order Creation & Checkout Validation

When a customer submits an order, the server does not rely on price calculations sent by the browser. Instead, the `orderController` queries the `Product` collection for every item, fetches the actual price (applying `discountPrice` if active), checks inventory stock levels, and dynamically computes the cost fields:

- **Items Price:** The sum of unit price times quantity for all lines in the order.
- **Shipping Fee:** Enforces a `FLAT_SHIPPING` charge of 200 PKR. If the Items Price exceeds the `FREE_SHIPPING_THRESHOLD` of 5000 PKR, the shipping is set to 0.
- **Tax Calculation:** Calculates flat tax at a `TAX_RATE` of 5% on the Items Price.
- **Total Price:** Sum of Items Price, Shipping, and Tax. Recorded on the new Order document.

Easypaisa & JazzCash Gateway Logic

For online card payments, the system integrates with Pakistani gateways Easypaisa and JazzCash. The checkout sequence is structured as follows:

- **Transaction Reference:** The server generates a unique gatewayTxnRef for the payment attempt.
- **Field Signing:** Using merchant credentials and a secret key, the server builds a request object and signs it using secure hashing algorithms (like MD5 or SHA-256).
- **Form Auto-Redirect:** The server returns this signed payload to the React client. The client renders an invisible, auto-submitted HTML form that POSTs the fields directly to the payment gateway's hosted page.
- **Return Webhook:** The customer enters their details on the gateway. The gateway redirects the browser back to `/api/payment/:method/return` with a hash signature. The server verifies the signature, flags the Order `isPaid` as true, updates `orderStatus` to 'Processing', decrements product stock, updates shop revenue counters, and sends out email alerts.
- **Fallback Simulator:** If gateway credentials are not configured, the server executes a fallback loop, finalizing payment immediately and routing the SPA to a checkout-success view.

7. Frontend Architecture & Redux Store

The my-project frontend client is structured as a Single Page Application (SPA). It uses Redux Toolkit for unified state caching and React Router DOM for route switching and role access control.

Redux Store Slice Configurations

The global Redux store is divided into modular slices, which manage the state lifecycle:

- **authSlice.js:** Caches active JWT, user profile metadata, role parameters, login status flags, error logs, and loading indicators.
- **productSlice.js:** Stores the platform-wide product catalog, categories, search queues, details of the selected product, and pagination cursors.
- **cartSlice.js:** Maintains items in the cart, quantities, and calculates client-side order subtotals.
- **orderSlice.js:** Handles the client's order histories, detail loads, and status updates.
- **notificationSlice.js:** Manages transactional messages and mark-as-read updates.
- **wishlistSlice.js:** Tracks saved products and syncs changes with backend endpoints.
- **adminSlice / sellerSlice.js:** Handles admin statistics and seller shop configurations.

Protected Routing & Navigation

Access controls on the client are enforced using a wrapper component, `<ProtectedRoute />`. This component reads the user's role from the `authSlice`. If the role doesn't match the required whitelist, the component redirects them to the login screen or a dashboard suited to their role, preventing cross-role access to dashboard views.

Dashboard Analytics & Styling System

For seller and admin dashboard reporting, the client uses Recharts to display interactive charts. Styling is implemented using Tailwind CSS utility classes and CSS variables in `index.css`, establishing a dark-theme dashboard design.

8. Appendix: Full API Endpoint Reference

The following table lists the complete REST API endpoints implemented on the backend Express server, mapping their HTTP verbs, route paths, role requirements, and description details:

Method	Route Path	Role Required	Description
POST	/api/auth/register	Public	Create customer or seller account
POST	/api/auth/login	Public	Local email/password login
POST	/api/auth/google	Public	Google Social OAuth login
POST	/api/auth/facebook	Public	Facebook Social OAuth login
GET	/api/auth/profile	Authenticated	Fetch active user profile details
PUT	/api/auth/profile	Authenticated	Update name, address, profile details
PUT	/api/auth/change-pass	Authenticated	Update local account password
POST	/api/auth/forgot-pass	Public	Request recovery email link
POST	/api/auth/reset-pass/:tok	Public	Reset password using email token
GET	/api/products	Public / Seller	List products (public) or seller list
GET	/api/products/:id	Public	Get product details & vendor info
POST	/api/products	Approved Seller	Add product to seller inventory
PUT	/api/products/:id	Seller / Admin	Update product fields or publish status
DELETE	/api/products/:id	Seller / Admin	Remove product from catalog
GET	/api/shops	Public	Retrieve list of all active shopfronts
GET	/api/shops/:id	Public / Seller	Get details and statistics of a shop
PUT	/api/shops/:id	Seller / Admin	Update name, description, banner, logo
POST	/api/orders	Customer	Create order & generate txn reference
GET	/api/orders/myorders	Customer	Retrieve active customer order log
GET	/api/orders/seller	Approved Seller	List orders containing seller items
GET	/api/orders/:id	Owner / Admin	Fetch detailed order invoice metrics
PUT	/api/orders/:id/status	Seller / Admin	Transition order shipment status
GET	/api/admin/stats	Admin	Fetch aggregate revenue and counts
GET	/api/admin/users	Admin	List users or pending seller profiles
DELETE	/api/admin/users/:id	Admin	Soft-deactivate user account
PUT	/api/admin/sellers/:id/appr	Admin	Approve seller request & email user
PUT	/api/admin/sellers/:id/reje	Admin	Reject seller onboarding application
GET	/api/admin/orders	Admin	Review platform-wide order log
GET	/api/categories	Public	Fetch all listing categories
POST	/api/categories	Admin	Create a new product category
DELETE	/api/categories/:id	Admin	Delete category from database
POST	/api/upload	Seller / Admin	Upload images (local or Cloudinary)
GET/POST	/api/payment/easypaisa/ret	Public	Easypaisa return callback page
GET/POST	/api/payment/jazzcash/ret	Public	JazzCash return callback page
GET	/api/payment/simulate	Developer	Simulate successful checkout completion

Note: Route files are secured on the Express server using `authMiddleware.js` (protect) and `roleMiddleware.js` (authorize/requireApprovedSeller). All incoming request payloads undergo JSON schema casting and validation to prevent injection or parameter tampering.